Laboratorio 2 · Parti e relazioni: chi possiede chi

Programmazione II (L-Z) · martedì 3 e giovedì 5 novembre 2026 · Visual Studio Code

Cosa allenate, e perché

Nella lezione 7 avete visto `equals`, i metodi booleani, i parametri di tipo classe, `null`, le enumerazioni, gli array come variabili di istanza e gli array di oggetti, e soprattutto le relazioni fra classi: l'associazione, l'aggregazione, la composizione, e il privacy leak che le minaccia tutte. Nella prima parte usate le nozioni una per una. Nella seconda costruite il circolo di calcio a 5 un passo alla volta: a ogni passo una relazione nuova, e siete voi a decidere chi possiede chi. Nella terza qualcuno prova a truccare la classifica, e scoprite che cosa si può prestare e che cosa no.

Il filo è sempre lo stesso: un oggetto che consegna le sue parti a chiunque le perde, perché chi le riceve può cambiarle (il problema); per ogni relazione si decide chi possiede chi, e quindi che cosa si condivide e che cosa no (la scelta); il costo sono le copie, più tempo e più memoria, e va pagato solo dove la relazione lo chiede (il prezzo).

Parte	Esercizio	Come	Tempo
1 · Le nozioni	Esercizio 1 · L'atleta	a coppie	45 minuti
1 · Le nozioni	Esercizio 2 · Prevedi l'output	da soli	15 minuti
2 · Problem solving	Esercizio 3 · Chi possiede chi	a coppie	120 minuti
3 · Privacy leak	Esercizio 4 · La classifica truccata	a coppie	45 minuti
Chiusura	I perché	tutti insieme	15 minuti

Martedì la parte 1 e i passi 1 e 2 dell'esercizio 3. Giovedì il passo 3, la parte 3 e la chiusura.

Prima di cominciare. Scaricate lo zip del codice e scompattatelo: ottenete la cartella `codice`. Ogni esercizio ha la sua cartella: apritela in Visual Studio Code (*File › Apri cartella*). Per ogni esercizio trovate anche i comandi per compilare ed eseguire da terminale.

Parte 1 · Le nozioni

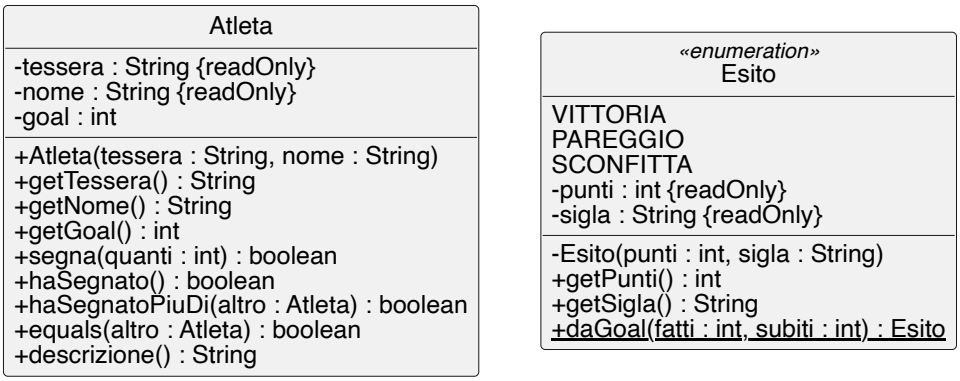
Esercizio 1 · L’atleta

⏱ 45 minuti · a coppie · cartella `esercizio1`

```
javac *.java
java ProvaAtleta
```

Scenario

Un circolo di calcio a 5 tiene l’archivio dei suoi atleti. Ogni atleta ha una tessera, per esempio `T-07`, che lo identifica e non cambia più, un nome e il numero di goal segnati in stagione. Due atleti con la stessa tessera sono lo stesso atleta, anche se sono stati registrati due volte. Di ogni incontro conta l’esito, che vale punti in classifica: 3 la vittoria, 1 il pareggio, 0 la sconfitta.



La classe Atleta e l’enumerazione Esito

Nel diagramma + vuol dire `public` e − `private`, `{readOnly}` vuol dire `final` e i membri sottolineati sono `static`.

Consegna

★ **La classe e i suoi metodi booleani** (25 minuti). Dentro `esercizio1` scrivete la classe `Atleta` del diagramma:

- `segna(quanti)` aggiunge i goal e restituisce `true`; se `quanti` non è positivo restituisce `false` e non cambia niente;
- `haSegnato()` dice se l’atleta ha almeno un goal, `haSegnatoPiuDi(altro)` se ne ha più dell’altro;
- `equals(altro)`, con il parametro di tipo `Atleta` come nella slide 5, dice se due atleti hanno la stessa tessera, e con `null` restituisce `false` invece di fermare il programma;
- `descrizione()` restituisce per esempio `T-07 Marco Neri: 2 goal`.

Verifica: `ProvaAtleta.java` deve compilare e stampare esattamente

```
T-07 Marco Neri: 0 goal
segna 2: true
segna 0: false
segna -1: false
T-07 Marco Neri: 2 goal
stessa tessera: == false, equals true
tessera diversa: equals false
confronto con null: false
Luca ha segnato: false
Marco più di Luca: true
```

★★ **L'esito come enumerazione** (20 minuti). Scrivete l'enumerazione `Esito` del diagramma: ogni valore ha i suoi punti e la sua sigla (`V`, `N`, `P`), assegnati dal costruttore privato (slide 67). Il metodo di classe `daGoal(fatti, subiti)` restituisce l'esito di un incontro dal punto di vista di chi ha fatto `fatti` goal.

Verifica: in `ProvaAtleta.java` togliete il commento `/* … */` intorno alla parte ★★. In fondo all'output devono comparire esattamente

```
3 a 1: VITTORIA (V), punti 3
2 a 2: PAREGGIO (N), punti 1
0 a 4: SCONFITTA (P), punti 0
esiti possibili: 3
```

Se sei bloccato

- Che cosa rende uno stesso atleta: il nome, la tessera, o il fatto di essere lo stesso oggetto?
- `marco.equals(null)`: che cosa succede se nel corpo scrivete `altro.tessera` prima di aver controllato `altro`?
- Due stringhe si confrontano con == o con `equals`?
- Un'enumerazione può avere attributi e un costruttore: chi lo chiama, e quante volte?

Esercizio 2 · Prevedi l'output

⏱ 15 minuti · da soli · cartella `esercizio2`

```
javac Maglia.java Prevedi.java
java Prevedi
```

`Maglia.java` è la maglia di un atleta, con un numero e due colori in un array; `Prevedi.java` è un `main` con sei righe di stampa e tre metodi che ricevono una maglia o un numero. Qui non si scrive codice: si legge.

★ **La previsione** (10 minuti). Senza eseguire niente, scrivete su carta che cosa stampa ogni riga, e perché.

★★ **Il confronto** (5 minuti). Eseguite `Prevedi` e segnate le differenze. Per ogni differenza scrivete in una riga quale idea vi ha tratto in inganno.

Se siete bloccati: un metodo riceve l'oggetto o una copia del riferimento (slide 14)? E se assegna al parametro un oggetto nuovo? Che cosa restituisce `getColori()`?

Parte 2 · Problem solving: chi possiede chi

Esercizio 3 · Chi possiede chi

⏱ 120 minuti · a coppie · cartella `esercizio3`

Scenario

Il circolo ha più squadre: la Under 21 e la Serie C. Ogni squadra gioca in un impianto, ha una rosa di al più 12 atleti e un calendario di incontri. L'impianto è del comune: più squadre ci giocano, e se una squadra si scioglie l'impianto resta, e non è una sua parte. Gli atleti sono del circolo, non della squadra: la squadra è fatta dei suoi atleti, ma uno stesso atleta può giocare in due squadre, e se una squadra si scioglie i suoi atleti restano. Gli incontri invece sono della squadra: nascono nel suo calendario, appartengono a lei sola e senza di lei non hanno senso. Costruite le classi in tre passi, una relazione per passo; come realizzare ogni relazione in Java lo decidete voi.

Come si lavora. Create in `esercizio3` la cartella `circolo` e copiateci `Atleta.java` ed `Esito.java` dell'esercizio 1, con `package circolo;` in testa. `App.java` è l'app del circolo, fuori dal package: non si modifica, tranne per togliere i commenti dei passi e provare le righe del muro.

```
javac App.java circolo/*.java
java App
```

Il muro. Alcuni passi hanno delle righe da aggiungere in fondo al `main` di `App`, una alla volta: le ✗ non devono compilare, le ✓ sì. Dopo la prova la riga si cancella.

Il registro delle scelte. Alla fine di ogni passo scrivete in due o tre righe, su carta o in un commento in testa ad `App`, quale relazione avete realizzato, con quale attributo, e chi crea gli oggetti che lo riempiono.

Passo 1 · L'associazione ★ (25 minuti)

Scrivete la classe `Impianto`, con un nome e una capienza, e la classe `Squadra`, con un nome e l'impianto dove gioca. L'impianto è un oggetto che esiste già: lo si passa al costruttore, e lo si può cambiare con `setImpianto`. `Impianto.descrizione()` restituisce per esempio `Palazzetto Nord (300 posti)` e `Squadra.descrizione()` `Under 21 gioca in Palazzetto Nord (300 posti)`. La capienza si può cambiare, per i lavori, con `setCapienza`.

Chi	Deve poter	Non deve poter
`App`	creare un impianto e le squadre che ci giocano, cambiare la capienza e l'impianto di una squadra, leggere le descrizioni	
`Squadra`	leggere la descrizione del suo impianto	

Verifica: `App` deve stampare esattamente

```
Under 21 gioca in Palazzetto Nord (300 posti)
Serie C gioca in Palazzetto Nord (300 posti)
Under 21 gioca in Palazzetto Nord (250 posti)
Serie C gioca in Palazzetto Nord (250 posti)
Under 21 gioca in Palestra Sud (120 posti)
Serie C gioca in Palazzetto Nord (250 posti)
il palazzetto c'è ancora: Palazzetto Nord (250 posti)
```

Il comune cambia la capienza del palazzetto, e le due squadre lo vedono: perché? E quando la Under 21 cambia impianto, perché la Serie C no? Un impianto è una parte della squadra?

Passo 2 · L'aggregazione ★★ (40 minuti)

Ogni squadra ha una rosa: l'array `rosa`, di al più `MAX_ATLETI` atleti, con la costante a 12, e il contatore `numeroAtleti` di quelli che ci sono. `aggiungi(atleta)` lo mette in rosa e restituisce `true`; restituisce `false`, senza cambiare niente, se l'atleta è `null`, se la rosa è piena o se c'è già un atleta con la stessa tessera. `togli(tessera)` lo toglie senza lasciare buchi nell'array. `elenco()` restituisce per esempio `Serie C: T-07 Marco Neri (3), T-03 Andrea Russo (0)`, con i goal di ogni atleta fra parentesi, e `Serie C: nessun atleta` se la rosa è vuota; `goal()` è la somma dei goal della rosa.

Chi	Deve poter	Non deve poter
`App`	aggiungere e togliere atleti, leggere elenco e goal, mettere lo stesso atleta in due squadre, leggere la costante `MAX_ATLETI`	cambiare la rosa senza passare da `aggiungi` e `togli`
`Squadra`	confrontare gli atleti per tessera	

Decidete voi: la squadra tiene gli atleti che riceve o una loro copia? Che cosa dice lo scenario? Un atleta, a differenza dell'impianto, è un membro della squadra: che cosa fa la squadra con la rosa che non fa con l'impianto?

Verifica: togliete il commento intorno alla parte del passo 2 di `App`. Dopo le righe del passo 1 devono comparire esattamente

```
Marco nella Under 21: true
Luca nella Under 21: true
Marco, di nuovo: false
Marco nella Serie C: true
Andrea nella Serie C: true
Under 21: T-07 Marco Neri (3), T-11 Luca Bianchi (0)
Serie C: T-07 Marco Neri (3), T-03 Andrea Russo (0)
goal di squadra: Under 21 3, Serie C 3
tolgo T-11: true, di nuovo: false
Under 21: T-07 Marco Neri (3)
```

Il muro:

- ✗ `u21.rosa[0] = null;`
- ✗ `u21.numeroAtleti = 0;`
- ✓ `System.out.println(Squadra.MAX_ATLETI);`

Passo 3 · La composizione ★★★ (55 minuti)

Ogni squadra ha il suo calendario, della classe `Calendario`, e il calendario ha i suoi incontri, della classe `Incontro`: per ogni incontro la giornata, l'avversario, i goal fatti e i goal subiti, e l'esito che ne viene. Un incontro si crea con `Incontro(giornata, avversario, goalFatti, goalSubiti)` e ha `descrizione()`, che restituisce per esempio `giornata 3: Gabbiani 4–0 (V)`. Una squadra registra un incontro con `registra(giornata, avversario, fatti, subiti)`, che restituisce `false`, senza cambiare niente, se quella giornata è già registrata, se i goal sono negativi o se il calendario è pieno. `punti()` somma i punti degli esiti, e `riepilogo()` restituisce per esempio `Serie C: 7 punti in 3 incontri (2 V, 1 N, 0 P)`. Il calendario tiene gli incontri in un array di 30 posti.

Chi	Deve poter	Non deve poter
`App`	registrare incontri, leggere punti e riepilogo	creare un calendario o un incontro, dare a una squadra un calendario fatto altrove
`Squadra`	creare il suo calendario e chiedergli punti, numero di incontri e conteggi per esito	
`Calendario`	creare i suoi incontri e leggerne giornata ed esito	

Decidete voi: quali classi sono `public`? Chi crea il calendario, e quando? Chi crea gli incontri? Che visibilità hanno i loro costruttori? Rileggete la regola di non condivisione (slide 44) e la composizione in Java (slide 45).

Verifica: togliete il commento intorno alla parte del passo 3 di `App`. Dopo le righe dei passi 1 e 2 devono comparire esattamente

```
giornata 1: true
giornata 2: true
giornata 2, di nuovo: false
giornata 3: true
giornata 4, goal negativi: false
Serie C: 7 punti in 3 incontri (2 V, 1 N, 0 P)
```

Il muro:

- ✗ `new circolo.Calendario();`
- ✗ `new circolo.Incontro(5, "Falchi", 1, 0);`
- ✓ `System.out.println(serieC.punti());`

Se sei bloccato

- L’oggetto esiste prima della squadra, o nasce dentro la squadra? Lo vede qualcun altro?
- Se la squadra si scioglie, che cosa succede a quell’oggetto nel mondo del circolo?
- Due squadre possono avere lo stesso atleta? E lo stesso incontro?
- Come si toglie un elemento da un array senza lasciare un buco? Pensate agli elementi che vengono dopo.
- Per dire se un atleta è già in rosa avete già un metodo: quale? E `togli` riceve solo la tessera: come si trova l’atleta?
- Una classe senza `public` davanti: chi la vede? E un costruttore senza modificatore?
- L’esito di un incontro va memorizzato o si può calcolare dai goal ogni volta che serve?

Parte 3 · Privacy leak: che cosa si presta

Esercizio 4 · La classifica truccata

🕛 45 minuti · a coppie · cartella `esercizio4`

```
javac App.java circolo/*.java
java App
```

Scenario

La federazione vuole leggere gli incontri e la rosa di ogni squadra, e il circolo le dà due metodi. Il primo giorno qualcuno, dall'app, prova a cambiare un pareggio in una vittoria. L'arbitro, invece, a volte deve davvero correggere un risultato. E la federazione, quando legge la rosa, deve poter aggiornare i goal di un atleta, ma non togliere un atleta dalla squadra.

Come si lavora. Create la cartella `esercizio4` e copiateci la cartella `circolo` dell'esercizio 3, finito fino al passo 3, e `App.java` di `codice/esercizio4`. È l'`App` di prima, con le righe dell'esercizio 3 già tutte attive e sotto le parti nuove. `App` non si modifica, tranne per togliere i commenti dei passi e provare le righe del muro, e a ogni passo le righe dell'esercizio 3 devono stampare esattamente come alla fine dell'esercizio 3.

Passo 1 · Gli incontri in lettura ★ (15 minuti)

Aggiungete a `Squadra` il metodo `getIncontri()`, che restituisce gli incontri giocati in un array. Prima scrivetelo nel modo più semplice, restituendo l'array del calendario, togliete il commento intorno alla parte del passo 1 di `App` ed eseguite: che cosa vedete? Poi correggetelo (slide 63).

Decidete voi: per leggere un incontro da fuori, che cosa deve diventare `public`, e che cosa resta friendly?

Verifica: dopo la correzione, dopo le righe dell'esercizio 3 devono comparire esattamente

```
incontri visti: 3
Serie C: 7 punti in 3 incontri (2 V, 1 N, 0 P)
```

Passo 2 · L'arbitro corregge ★★ (15 minuti)

L'arbitro corregge il risultato di una giornata con `correggi(giornata, fatti, subiti)` della squadra, che restituisce `false` se la giornata non c'è o se i goal sono negativi. I goal di un incontro, quindi, non sono più fissi, e dopo la correzione anche l'esito è quello giusto. Per correggere, l'incontro ha il metodo `correggi(goalFatti, goalSubiti)`, che il calendario chiama per conto della squadra. Chi ha in mano un incontro letto con `getIncontri()` non deve poterlo correggere.

Decidete voi: la copia dell'array basta ancora, o servono anche le copie degli incontri? C'è un modo di non copiarli?

Verifica: togliete il commento intorno alla parte del passo 2. Dopo le righe del passo 1 devono comparire esattamente

```
correzione della giornata 2: true
correzione della giornata 7: false
Serie C: 9 punti in 3 incontri (3 V, 0 N, 0 P)
giornata 2: Delfini 3–2 (V)
```

Il muro:

- ✗ `serieC.getIncontri()[0].correggi(9, 0);`

Passo 3 · La rosa in prestito ★★★ (15 minuti)

Aggiungete `getRosa()`, che restituisce gli atleti della rosa in un array lungo quanto la rosa. Chi lo riceve non deve poter togliere o sostituire un atleta della squadra, ma i goal che fa segnare a un atleta devono valere anche per la squadra.

Decidete voi: che cosa si copia, l'array, gli atleti, tutti e due o niente? Che cosa dice l'aggregazione?

Verifica: togliete il commento intorno alla parte del passo 3. Dopo le righe del passo 2 devono comparire esattamente

```
atleti nella copia: 2
Serie C: T–07 Marco Neri (3), T–03 Andrea Russo (1)
atleti in rosa: 2
```

Se avete tempo, provate anche a copiare gli atleti: che cosa cambia nell'output, e perché è sbagliato?

Se sei bloccato

- Un array è un oggetto: chi riceve il riferimento all'array interno, che cosa può fare?
- Quante celle ha l'array del calendario, e quante ne sono piene?
- Copiare l'array copia anche gli oggetti a cui le celle si riferiscono?
- Un metodo che deve poter chiamare il calendario ma non chi sta fuori dal package: `private`, friendly o `public`?
- Un atleta della rosa è una parte privata della squadra o un oggetto del circolo che la squadra condivide?

Chiusura · I perché

⏱ 15 minuti · tutti insieme

Non è un test: sono domande da discutere, ciascuna su una scelta delle due giornate.

1. Perché `equals` confronta la tessera e non il nome? Che cosa succederebbe con due Marco Neri nel circolo?
2. Che cosa sarebbe cambiato se `Esito` fosse stato un `int` (0, 1, 3) o una `String`? Che cosa vi ha evitato l'enumerazione, e che cosa costa?
3. Nel passo 2 la squadra tiene gli atleti che riceve, non una copia. Che cosa sarebbe successo ai goal di Marco con una copia, e quale riga dell'output lo mostra?
4. L'impianto e gli atleti sono tutti oggetti che la squadra riceve da fuori e che esistono senza di lei. Eppure l'impianto è un'associazione e la rosa un'aggregazione: in Java il codice è lo stesso, un riferimento a un oggetto creato fuori, quindi la differenza non esiste. *(trova l'errore)*
5. Il calendario nasce dentro la squadra, e nessun metodo lo restituisce. Eppure `getIncontri()` restituisce gli incontri, che sono parti del calendario: quindi un riferimento è uscito, e per questo la relazione è diventata un'aggregazione. *(trova l'errore)*
6. Nel passo 2 dell'esercizio 4 avete copiato gli incontri, o avete reso friendly `correggi`? Che cosa costa ciascuna delle due strade?
7. `getRosa()` restituisce una copia dell'array con gli stessi atleti dentro. Eppure la slide 36 dice che, per evitare il privacy leak, un metodo `get` deve restituire una copia dell'oggetto: quindi `getRosa()` dovrebbe copiare anche ogni atleta. *(trova l'errore)*